



AI-Chatbot Development based on the Large Language Model Deepseek

Comparison of Retrieval Augmented Generation and Fine-Tuning

Bachelors Thesis

Author: Tobias Brandner

StudentID: 12215979

Computer Science

Supervisor: O.Univ-Prov. Dipl.-Ing Dr. Wolfgang Pree

Submission Date: July 16, 2025

Department of Computer Science
University of Salzburg

Abstract

In recent years, large language models (LLMs) have become central to natural language processing (NLP) applications. This thesis explores and compares two prominent approaches to customizing LLM outputs for chatbot development: fine-tuning and Retrieval-Augmented Generation (RAG). Using the open-source DeepSeek-R1 family of models, both methods are implemented and benchmarked, aiming to replicate the performance of the existing Brother-David Bot, a project by the software engineering team at the Paris Lodron University of Salzburg (PLUS), on consumer hardware.

The fine-tuning approach involved training smaller, distilled DeepSeek models using responses generated by the original chatbot. In contrast, the RAG pipeline enriched user prompts with received context before passing them to the LLM. Performance was evaluated through multiple metrics, including semantic similarity, language consistency, and inference time.

Results show that fine-tuned models achieve comparable quality to the original chatbot, especially in mimicking the original chatbots style. However, trained models struggle with hallucinations, making up quotes and lessons on the spot. This work offers insights into the trade-offs between adaptability and efficiency in LLM-based chatbot design and highlights the potential of local, fine-tuned models as viable alternatives to cloud-based solutions.

Contents

1	Introduction	4
2	Large Language Model (LLM) Basics	5
2.1	Key Terminology	5
2.2	How LLM Inference works	8
2.3	DeepSeek LLMs	9
2.3.1	Mixture-of-Experts (MoE) architecture	9
2.3.2	The training of DeepSeek-R1	9
2.3.3	Knowledge Distillation	10
3	Adapting LLM output after post-training	12
3.1	Retrieval Augmented Generation (RAG)	12
3.2	Fine-Tuning	13
3.3	Other methods	14
4	Introduction to the practical project	15
4.1	Brother-David Bot	15
4.2	Research goals	16
4.3	Tools and Applications Used	16
5	Implementation	18
5.1	LLM Interaction	19
5.2	Training	22
5.3	Benchmarks & Metrics	28
6	Experimental setup	29
6.1	Examined DeepSeek variants	29
6.2	Benchmark Tests	29
6.3	Metrics for performance evaluation	30
7	Results	32
7.1	Time and Token analysis	32
7.2	Quality-based analysis	35
7.3	Analysis of the results	39
8	Conclusion	41

1 Introduction

In the past few years we have seen remarkable advancements in the field of natural language processing (NLP), and especially in large language models (LLMs). While a few years back AI generated content was mostly seen as a gimmick, modern models like OpenAI's ChatGPT or Google's Gemini demonstrate an unprecedented ability to understand and generate human language.

One of the latest competitors in the growing ecosystem of LLMs is DeepSeek, whose entry took the world by surprise. Not only because it demonstrated that powerful LLMs could be developed outside of the USA, but also because of its significantly more cost-efficient design and its open-source availability. This meant that everyone with access to powerful enough hardware could theoretically run the model locally. In addition to the full DeepSeek-R1 model, however, a few so called 'distilled' versions of DeepSeek-R1 were also released. These smaller, more efficient variants are specifically optimized for consumer hardware and can be fine-tuned with the help of modern tools, making advanced NLP technologies more attainable for a wider audience.

This bachelor's thesis will explore the inner workings of LLMs and evaluate two key approaches in chatbot development: fine-tuning and Retrieval-Augmented Generation (RAG). As a practical part, this work will then attempt to fine-tune a chatbot based on the existing in-house "Stefan Zweig Chatbot" using locally available hardware. The outcomes of this fine-tuning process will be compared to a RAG-based implementation, with the goal of analyzing the strengths, limitations, and practical implications of each method.

2 Large Language Model (LLM) Basics

To properly understand the different techniques used in chatbot creation, it is important to have a basic understanding of how LLMs function. This chapter will delve into the inner workings of Large Language Models and explain their architecture and methodologies.

2.1 Key Terminology

This chapter introduces key terms and concepts in the realm of LLMs that are important for understanding methods later used in this thesis.

Tokens

LLMs treat text not as a sequence of words, but as a sequence of *tokens*. Tokens are the result of text being broken down into smaller segments and can range from multiple words to individual letters and anything in between. Being aware of the tokenization process is crucial as it forms the input and output vocabulary of the model and can differ between different LLM implementations. [13]

Embedded tokens

To be usable by a computer system, tokens have to be converted into a numerical form. This process is called *embedding* and turns every token into an n-dimensional vector, where n usually is a big number and depends on the embedding algorithm.

It is important to note that these embeddings do not take into account the *context* a token appears in, e.g. words with multiple meanings all have the same vector embedding regardless of context.

Transformer

The *transformer* is the neural network architecture that is the foundation of modern LLMs. First introduced in the paper "*Attention is All You Need*" in 2017 [15], the transformer architecture involves a two-step process responsible for building a contextualized representation out of embedded tokens. In the first step it computes a value for each token that corresponds not only to itself, but also to the context surrounding it. This works by employing the so-called *attention-mechanism* where, through mathematical calculations, related embedded tokens get influenced and changed by each other.

In the second step, the resulting vectors get passed through a more traditional feed-forward network adding additional learned context to them. These two steps often get processed multiple times in parallel and also potentially thousands of times in succession, producing a heavily context-enriched final list of output tokens.

Pre-training

Pre-training is the most data- and computational power intensive part of training an LLM. It also marks the first step of training, where the model learns syntax and semantics of a language and builds up a general knowledge-base. The goal of pre-training for generative AI is often language modeling, where the model is trained to predict the next token given a sequence of tokens [14].

Post-training

Post-training is a relatively new term in the realm of LLMs and has varying definitions across the literature. [12][14].

In the context of this work, post-training will simply be considered as the step after pre-training, where the pre-trained base model gets refined to fit a specific task. Techniques used for post training include, but are not limited to, *Reinforcement Learning* and *Fine-Tuning*. A finished post-trained model may still be further refined by Fine-Tuning.

Parameters

Parameters of an LLM refer to weights and biases of the neural network that are learned during the training process. The modification of these parameters define the output of the LLM during inference.

Inference

The term *Inference* is used when using a trained LLM to generate output for a specific prompt, e.g. asking a question to ChatGPT or DeepSeek.

User Prompt

The *user prompt* refers to the explicit input provided by the user to the LLM during inference. It typically contains a question, instruction, or conversational message that the model is expected to respond to.

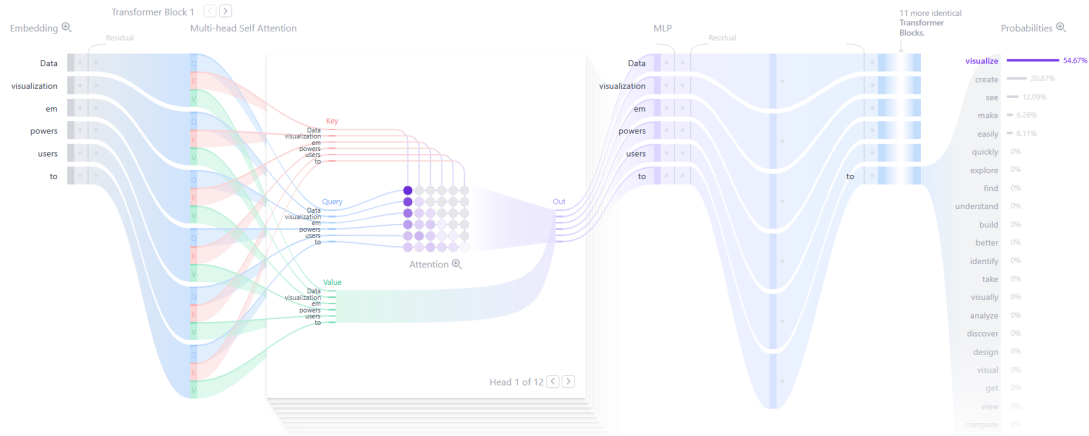


Figure 1: The path the prompt takes through the transformer, visualized [4]

System Prompt

The *system prompt* is an additional type of input not provided by the user and used to guide the behavior of the LLM when the user provides their input. Most of the time it is invisible to the user and can define the model’s role (e.g., “You are a helpful assistant”) or set instructions for how it should respond to the user prompt.

Context Window

The *context window* refers to the maximum length of the input sequence that an LLM can process at once. This limit affects how much preceding text the model can consider when generating the next token. The size of the context window varies greatly between different LLMs.

Consumer Hardware

While not a key term in the realm of LLMs in a traditional sense, the term *consumer hardware* is very important for this thesis. In this thesis, consumer hardware refers to physical components designed primarily for use by individual consumers. LLMs are usually run on systems equipped with multiple expensive GPUs like the Nvidia A100, while the local models used in this thesis are employed on a server with one RTX 3090 GPU.

2.2 How LLM Inference works

The centerpiece of Large Language Models is the *Transformer* architecture. A Transformer is a kind of neural network specialized in predicting the next token given a set of input tokens. This seemingly simple task of 'next token prediction' is remarkably powerful and enables LLMs to generate coherent replies.

During the training phase of an LLM, the LLM is presented with large amounts of text data, learning statistical patterns between words, phrases and sentences. An LLM does not "understand" language in the way a human does, but rather builds an internal representation of language that allows it to predict the next token given the preceding context. [15]

The Transformer Architecture

At the core of the transformer architecture lies the concept of *self-attention*. First introduced in the revolutionary paper "Attention is All You Need" from 2017, self-attention allows the model to weigh the importance of different tokens relating to each other. With key, query and value matrices, each token's *embedding* is modified to more closely represent its real meaning in the context of tokens that came before it.

Following the self-attention block, the context-enriched tokens get passed through a highly parallel feed-forward neural network designed to further refine token embeddings with more nuanced learned representations. This block, again, works with matrix multiplications.

In LLMs, the embedded tokenized input tokens are usually fed through many layers of transformer blocks, with each layer consisting of multi-head self-attention blocks and a feed-forward neural network. The final transformer layer outputs a vector, which is then passed through a *softmax function* to produce a probability distribution over the model's vocabulary for the next token. Depending on the sampling method, and hyperparameters such as temperature, one predicted token is chosen and appended to the input text. This process can then be repeated until a predefined stopping condition, such as generating an "end-of-sequence" token, is met. [15]

Figure 1 shows a visualization of one head and layer of a Transformer. The most important takeaway of this chapter is that the next predicted token is largely dictated by two factors:

1. The user input, possibly enriched with further context by an optional system-prompt
2. The learned weights and biases of the neural network architecture, including both matrices of the self-attention as well as the feed-forward layers.

2.3 DeepSeek LLMs

This chapter provides an overview of the DeepSeek family of Large Language Models, focusing particularly on architectural innovations that separate it from other LLMs mentioned so far.

DeepSeek-AI, a China-based startup focussing on LLMs, has developed a series of LLMs with a focus on improved reasoning capabilities and model parameter scaling. DeepSeek-AI’s flagship LLM, DeepSeek-R1, as well as many other of their models employ an alternative strategy, the so-called Mixture-of-Experts (MoE) architecture specifically designed to combat the extensive computational overhead when scaling LLMs. This architecture, as well as the best DeepSeek models and further benefits of ”Model Distillation” will be explained below. [10]

2.3.1 Mixture-of-Experts (MoE) architecture

Scaling the parameter count and amount of data LLMs are trained with has led to significant progress in natural language processing. However, with significant scaling of the number of active parameters in a model also comes significant computational overhead.

The Mixture-of-Experts (MoE) architecture aims to solve this ever-growing problem by splitting available parameters into N experts and activating only the top- K of them, lowering the total amount of active parameters significantly [7].

Potential benefits of the MoE architecture have been well documented even before the emergence of DeepSeek, with one paper from 2022 demonstrating a performance increase in comparison to GPT-3 while lowering the energy consumption for training by 2/3 [8].

2.3.2 The training of DeepSeek-R1

The flagship model of DeepSeek-AI, DeepSeek-R1 made significant progress in the realm of LLMs by achieving powerful reasoning and general LLM qualities through a new training methodology.

Fine-Tuning the base model: In the first step towards DeepSeek-R1, its base model, the pre-trained DeepSeek-V3, gets fine-tuned with thousands of so-called *cold-start data*. This cold-start data consists of LLM-generated and human-refined prompt-response pairs, selected and edited to provide long Chain-of-Thought responses [10]. This additional step before the post-training is added to provide a more accurate neural network from the start and to incentivize the correct formatting of responses.

Post-training pipeline: The post-training pipeline consists of two steps, Group Relative Policy Optimization (GRPO) and Supervised Fine-Tuning, which are applied alternating.

GRPO is a cost-efficient Reinforcement-Learning (RL) Algorithm. The RL step of the post-training aims to help the model to develop reasoning capabilities without any supervised data. Accuracy of the response, e.g. for mathematical calculations, adhering to the defined response-template and language consistency get rewarded and lead to updating of model weights.

In the second step, the resulting model is used to generate data for Supervised Fine-Tuning through inference. This data is then sampled and refined, either through other LLMs or humans, and used to fine-tune the model.

The resulting model of these two steps is called a *Checkpoint* and saved. After thousands of iterations, a certain checkpoint was selected and named DeepSeek-R1. [10]

2.3.3 Knowledge Distillation

Although more resource-friendly than similar Large Language Models, DeepSeek-R1 still needs expensive equipment to run. With its 671 Billion Parameters, consumers would have to make a significant investment in hardware to run the model locally.

As a solution to this problem, as part of their research DeepSeek-AI demonstrated the efficiency of distilling smaller models based on more hardware-friendly architecture from the original DeepSeek-R1.

The core idea behind knowledge distillation is to train a smaller student model to mimic the behavior of a larger teacher model, rather than directly training it on the original dataset. This is typically accomplished by using the teacher model’s

Model	AIME 2024 pass@1	MATH-500 pass@1	GPQA pass@1
GPT-4o-0513	9.3	74.6	49.9
Claude-3.5-Sonnet-1022	16.0	78.3	65.0
OpenAI-o1-mini	63.6	90.0	60.0
QwQ-32B-Preview	50.0	90.6	54.5
DeepSeek-R1-Distill-Qwen-1.5B	28.9	83.9	33.8
DeepSeek-R1-Distill-Qwen-7B	55.5	92.8	49.1
DeepSeek-R1-Distill-Qwen-14B	69.7	93.9	59.1
DeepSeek-R1-Distill-Qwen-32B	72.6	94.3	62.1
DeepSeek-R1-Distill-Llama-8B	50.4	89.1	49.0
DeepSeek-R1-Distill-Llama-70B	70.0	94.5	65.2
DeepSeek-R1	79.8	97.3	71.5

Table 1: Comparison of DeepSeek-R1 distilled models and other comparable models on reasoning-related benchmarks. [10]

outputs as targets for the student model during its training.

The benefits of distillation are substantial, and include, but are not limited to, reduced computational cost, reduced model size and faster inference. Crucially, as demonstrated by DeepSeek-AI’s research shown in Table 1, distilled versions of models like DeepSeek-R1 still yield very good results on performance benchmarks. [10]

3 Adapting LLM output after post-training

As stated in chapter 2.2, the main factors defining the output of a specific Large Language Model are:

1. The input, consisting of user input and an optional system prompt
2. The parameters of the underlying neural network

This chapter will explain the two main methods of adapting LLM output after post-training, Fine-Tuning and Retrieval Augmented Generation. Both of these methods work fundamentally differently and seek to adapt the output by tweaking one of the factors mentioned above.

3.1 Retrieval Augmented Generation (RAG)

Retrieval Augmented Generation is a technique that aims to enhance the output of an LLM by adding retrievable knowledge to the system prompt. This allows the LLM to generate more accurate responses that can be influenced by information not provided by the user. [11]

The typical RAG pipeline involves the following three steps:

1. **Retrieval:** Given a user prompt, a retrieval system searches a database for information relevant to a given prompt. The exact retrieval mechanism can vary depending on the implementation, but commonly embedding models are used to perform a vector-similarity calculation on both input and documents.
2. **Augmentation:** Relevant documents or document snippets and potentially a system-prompt are added to the user-prompt.
3. **Generation:** The augmented prompt is fed into the LLM, which then generates a response, taking into account any relevant information added in the prior step.

Benefits and drawbacks of RAG

A huge benefit of developing chatbots using RAG lies in the relative simplicity of its implementation. Without the need of directly changing model parameters, one Large Language Model could be used in numerous applications.

RAG is also very adaptable, with a change in the retrieval system directly correlating to the LLM output. In this way, up-to-date information can easily be

incorporated into a chatbot.

The main drawback of RAG is the increased input generated for every single inference. Augmenting the prompt with retrieved documents can substantially lengthen the input sequence, impacting processing time and cost.

A second drawback is also the need for a retrieval system and database to store and manage relevant information. While generally negligible when compared to the computational cost of inference, the retrieval of information, especially when stored on a remote server, adds additional latency to every LLM interaction.

3.2 Fine-Tuning

In contrast to RAG, fine-tuning adapts the output of an LLM by directly modifying the parameters of the underlying neural network. This is done by taking a pre-trained or post-trained LLM and further training it on a task-specific dataset. Most often, fine-tuning happens by presenting the LLM with input-output pairs and adapt parameters to guide LLM responses to closer resemble said outputs.

Benefits and drawbacks of fine-tuning

Compared to RAG, fine-tuning could potentially significantly lower inference costs of an LLM, as no documents would have to be appended to the user prompt. Additionally, no separate database and retrieval system would be needed.

However, fine-tuning Large Language Models presents its own set of challenges. Fine-tuning can be a computationally intensive and time-consuming process, especially with models with billions of parameters. Additionally, up-to-date information cannot easily be incorporated into the LLM and fine-tuned models can only be used in the context they were adapted for.

While the computational and storage cost of fine-tuning can be partially mitigated by using certain techniques like Low-Rank Adaptation (LoRA), which focuses on training a specific subset of parameters only, there is no easy solution to incorporating new information into a fine-tuned LLM without RAG. For this reason, fine-tuning should only be done after careful consideration of a project's goals. [16]

3.3 Other methods

While RAG and fine-tuning are the main focus of this work, there are several other methods of adapting LLMs that are frequently used.

Knowledge distillation

Like regular fine-tuning, knowledge distillation adapts an LLM by changing its model parameters. When performing knowledge distillation, a bigger and more complex model is transferred into a smaller one. This is also the method used by DeepSeek for generating the smaller DeepSeek-R1 models and leads to more computationally efficient LLMs.[14]

Prompt Engineering

Prompt engineering aims to guide LLMs towards generating desired responses by carefully designing and optimizing the instructions given to the LLM. It is similar to RAG in the sense that the input prompt is augmented, but crucially lacks the retrieval component of RAG.

Prompt engineering has proven effective in practice but also has its limitations: As no additional knowledge is added to the LLM by either document retrieval or training, it is solely reliant on the LLM’s pre-trained knowledge. [9]

4 Introduction to the practical project

The practical part of this work is focused on training two distilled DeepSeek-R1 models on local hardware. First, the pre-existing Brother-David Bot will be introduced, which serves as a benchmark for the local models, after which used tools and applications will be briefly listed and relevant parts of the codebase will be presented.

4.1 Brother-David Bot

The Brother-David Bot is a project by the software engineering team at the Paris Lodron University of Salzburg.

Brother David Steindl-Rast

Brother David Steindl-Rast is a well-respected Benedictine monk, author and spiritual teacher. Growing up in Austria, he travelled to the US in 1953 to become a monk and participated in a Vatican mission to work on interreligious dialogue between Christianity and Buddhism. The core theme of Brother David's teachings revolves around "gratitude as a spiritual path". The now 98 years old monk currently lives at a monastery in Austria. [1]

Brother David's teachings as an AI Bot

Naturally, turning a living person into an AI chatbot raises ethical and moral questions. However, in the case of Brother David Steindl-Rast, this project is undertaken with his full knowledge and enthusiastic support. [1]

The Brother David Bot application offers several features to explore Brother David's philosophy and teachings [2]:

- Conversational AI: A chatbot designed to answer questions according to Brother David's teachings.
- Quotes and other information about Brother David can be viewed on the homepage.
- Voice generation: Chatbot answers and quotes can be played in an AI-generated voice that closely resembles that of Brother David himself.
- Gratitude Diary: Users can record what they are grateful for on a daily basis.

Implementation of the Brother David Bot

The chatbot part of the Brother David Bot is implemented via the ChatGPT API and RAG. When a user prompt is sent from the application to the LLM, it first gets modified with Retrieval Augmented Generation: A fixed system prompt guides the model towards a general tone, and relevant documents queried from a database are appended to the prompt. A response is then generated using the ChatGPT API call and displayed in the user’s application.

4.2 Research goals

The practical part of this work now aims to explore and experiment with local models to replicate the OpenAI Brother David Bot with a smaller, distilled DeepSeek model on consumer hardware.

The main research questions are as follows:

1. Can smaller, local models achieve performance comparable to ChatGPT?
2. Can pure fine-tuning be used as an alternative to RAG?
3. How relevant is the performance and latency overhead of RAG?

4.3 Tools and Applications Used

As a basis for the implementation, the Python programming language as well as a few LLM-related libraries were used:

Python

All modules for this project were implemented using Python. Python is a popular, high-level, general-purpose programming language known for its readability and versatility. It has a vast ecosystem of libraries, making it widely used in web development, data science, and most importantly for this work, machine learning and LLM training.

Ollama

Ollama is an open-source tool that allows users to easily download, run, and manage large language models directly on their local machines. It simplifies the process of interacting with various LLMs by providing a command-line interface and API. Ollama optimizes models for local execution through techniques like quantization, making powerful AI accessible on consumer hardware. Ollama can also be integrated directly into Python via its official library. [3]

Unsloth

Unsloth is a Python library designed to significantly accelerate and optimize the fine-tuning of large language models, particularly for tasks like Low-Rank Adaptation. It boasts remarkable improvements in training speed and memory efficiency compared to standard methods, enabling efficient LLM training on less powerful hardware. [5]

5 Implementation

In this chapter, the technical implementation of all modules needed for this work will be described in detail. All modules developed for this project can be categorized into one of the four following packages:

1. **LLM Interaction:** This package handles the communication with the LLMs. This includes, but is not limited to, managing inference by forwarding queries to Ollama, as well as handling model responses. Additionally, RAG calls to the provided API are done here.
2. **Server Actions:** In this package, connections with the remote server are handled. This also includes file transfer, executing code on remote and receiving system information. As this part of the code is not relevant for the goal of this work, and primarily serves administrative purposes, it will not be explained in detail.
3. **Training:** The training package encapsulates functionalities related to preparing fine-tuning data and executing the fine-tuning process. The most important part here is the Unsloth-based training code that is designed to efficiently fine-tune the LLM on self-sourced data.
4. **Benchmarks and Metrics:** In this final package, benchmarks are defined and implemented. It also encapsulates extensive tests of model performance and visualizations of their results.

5.1 LLM Interaction

The LLM Interaction Package is comprised of two python modules, `ollama_call.py` and `rag_call.py`

”Ollama call” module

```
from secrets_config.secret_variables import OLLAMA_HOST
from ollama import Client

client = Client(
    OLLAMA_HOST
)

def send_message(message, model='deepseek-r1:8b'):
    """
    Sends a chat request to the Ollama API using the provided client.

    Args:
        model (str): The model to use for generating responses.
        messages (list): A list of message dictionaries containing role and
            content.

    Returns:
        str: The content of the response message.
    """
    response = client.chat(
        model = model,
        messages = [
            {
                'role': 'user',
                'content': message,
            },
        ],
        options = {'num_ctx': 4096}
    )
    return response['message']['content']
```

Code Fragment 1: The general structure of calls to Ollama via Python

Code Fragment 1 shows how messages can be forwarded to Ollama using Python and the Ollama library. The function accepts two arguments: the message that should be sent, and the model that should process the message. The option *”num_ctx”: 4096* specifies that the LLM should work with a context length of

4096, as opposed to Ollama's standard of 2048. This has been changed to accommodate for RAG-augmented messages, which often fall into the 2500-3000 token range.

Additional functions in *ollama_call.py* include a special method that returns not only the response but also metrics to be used for benchmarking, and a *send_message* wrapper function that sends a RAG message to Ollama.

"RAG call" module

The RAG call module primarily handles interactions with two APIs: *get_rag_response(...)* returns a JSON string from the RAG API to be used when constructing a RAG message for the chatbot, while *get_openai_response(...)* returns the message from the original OpenAI-RAG-chatbot to a given prompt.

```
def get_rag_response(message, application="bruder-david", character="8",
    locale="de", source_num=2, quote_num=2, safeguard_num=2):
    """
    Constructs and calls the RAG API.

    Args:
        message (str)      : The user-prompt which should be augmented.
        application (str)  : The application for which RAG messages should be
                           returned.
        [...]

    Returns:
        JSON string returned by the API.
    """
    rag_call = ("curl '" + RAG_URL + "get-rag-data?"
        + "message=" + message.replace(" ", "_")
        + "&application=" + application
        + "&character=" + character
        + "&locale=" + locale
        + "&source_num=" + str(source_num)
        + "&quote_num=" + str(quote_num)
        + "&safeguard_num=" + str(safeguard_num)
        + "' -H 'Authorization: Bearer " + RAG_KEY + "'"

    stdin, stdout, stderr = server.execute_command_on_remote(rag_call)
    return stdout.read().decode().strip()
```

Code Fragment 2: The general structure of API calls

```

def build_system_prompt(message, application="bruder-david", character="8",
    locale="de", source_num=2, quote_num=2, safeguard_num=4):
    """
    Builds the full system prompt for a given message.

    Args:
        [...]

    Returns:
        Finished system prompt used for inference with an LLM.
    """
    rag_output = get_rag_response(message, application, character, locale,
        source_num, quote_num, safeguard_num)

    text_output = "References: \n" + get_texts(rag_output)

    safeguard_output = ("These are predefined responses to certain topics. If
        applicable, keep your answers as similar as possible to these examples:
        \n" + get_safeguards(rag_output))

    quote_output = ("Brother David likes to use quotes in his texts. If
        applicable, use insert one of these quotes in your answer (but do not
        use the same quote more than once): \n" + get_quotes(rag_output)
        + "Falls das Zitat von David Steindl-Rast ist sollst du nicht zitieren,
        denn du bist David Steindl-Rast!")

    question_prompt = "Beantworte folgende Frage: " + message + "\nAntworte
        unbedingt auf Deutsch!"

    system_prompt = "\n\n".join([SYSTEM_PROMPT_BASE, text_output,
        safeguard_output, quote_output, question_prompt])
    return system_prompt

```

Code Fragment 3: How a RAG prompt is built using the RAG API

Code Fragment 2 shows how RAG responses are queried using the API, while Code Fragment 3 shows how the user prompt is modified to form a RAG prompt. The method for building the system prompt is based directly on a version of the Brother David Bot from May 2025.

5.2 Training

The training package consists of *generate_data.py* and *unsloth_training.py*, where the first module generates training data in the form of input/output pairs and the second performs the fine-tuning of a specified DeepSeek model.

Generate Training Data

The training data generation module mainly consists of one function, *generate_rag_training_data(...)* which takes in four arguments:

- **file_path**: The path to a CSV file with a list of input questions (prompts)
- **prompt_count**: The number of prompts to process
- **variations**: The number of variations to generate per prompt
- **offset**: The starting index for prompt processing (default is 0)

The function processes the number of prompts specified by the arguments and generates a certain amount of output variations by prompting the specified RAG-augmented LLM (either DeepSeek or OpenAI-based). After enough input-output pairs are generated, they are saved to a CSV file in a standardized training set format. Code Fragment 4 shows the code that is run for every prompt.

```
rag_output = rag.get_open_ai_response(instruction)

new_entry = {
    "instruction": instruction,
    "input": input_text,
    "output": rag_output,
    "text": f"Es folgt eine Frage zu einem philosophischen Thema. Beantworte
           diese sinnvoll.\n\n### Frage: {instruction}\n\n### Antwort:
           {rag_output}"
}

augmented_data.append(new_entry)
```

Code Fragment 4: The code that is called for every input, in this case by prompting the OpenAI API

Unsloth Training

The Unsloth-training module is arguably the most important part of this work. It uses the data generated by the first module in this package to train a specified distilled DeepSeek variant.

```
from unsloth import FastLanguageModel
import torch

#### ---- MODEL ---- ####
available_models = ["unsloth/DeepSeek-R1-Distill-Llama-8B-unsloth-bnb-4bit",
                   "unsloth/DeepSeek-R1-Distill-Qwen-14B-unsloth-bnb-4bit"]

param_model = available_models[1]

## Model specific parameters ##
if param_model == "unsloth/DeepSeek-R1-Distill-Llama-8B-unsloth-bnb-4bit":
    print("Training Deepseek 8B Model")
    param_lora_alpha = 128          #64 for DeepSeek-R1:14b
    param_r = 128                  #64 for DeepSeek-R1:14b
    param_max_seq_length = 4096
    param_batch_size = 2           #1 for DeepSeek-R1:14b
    param_gradient_accumulation_steps = 8
```

Code Fragment 5: Parameters for training

Code Fragment 5 shows how the training parameters for the following fine-tuning process are defined. It is important to note that certain parameters differ when using DeepSeek-R1:14b, as this model takes up more memory than the 8b version by default, and thus memory has to be saved through shrinking the LoRA alpha and rank as well as the batch size as not to exhaust the system's available VRAM and RAM.

```
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = param_model,
    max_seq_length = param_max_seq_length,
    dtype = torch.bfloat16,
    load_in_4bit = True,
)
```

Code Fragment 6: Quick setup of model and tokenizer using the FastLanguage-Model module

```

model = FastLanguageModel.get_peft_model(
    model,
    target_modules =
        ["q_proj", "v_proj", "k_proj", "o_proj", "gate_proj", "down_proj", "up_proj"],
    lora_alpha=param_lora_alpha,
    lora_dropout=0,
    r=param_r,
    bias = "none",
    use_gradient_checkpointing = "unsloth",
    random_state = 42,
    use_rslora = False,
    loftq_config = None,
)

```

Code Fragment 7: Quick setup of model and tokenizer using the FastLanguageModel module

Code Fragments 6 and 7 show the initialization of our model. With the *FastLanguageModel.from_pretrained(...)* function, we load a pre-trained large language model. The `model_name` parameter specifies which model to load and `max_seq_length` defines the maximum context window size the model can handle during training.

The `d_type` and `load_in_4_bit` parameters enable different optimizations for the model: The data type is set to a 16-bit float, and the weights are even loaded in 4-bit quantization. This significantly reduces GPU memory usage while only marginally lowering model precision.

The function also returns the model's tokenizer, which will later be applied to our custom training data to prepare it for the fine-tuning process.

```
from datasets import load_dataset
from unsloth import to_sharegpt

dataset = load_dataset('json', data_files=file_path, split = "train")

# Convert the dataset to ShareGPT format
dataset = to_sharegpt(
    dataset,
    merged_prompt = "instruction",
    output_column_name = "output"
)
```

Code Fragment 8: Importing of the generated dataset, taken from `unsloth_training.py`

As Code Fragment 8 shows, the data is first imported from the json and labeled as the train split. The `to_sharegpt(...)` function then extracts the relevant columns from each entry in the dataset and converts them into the ShareGPT format required for training. Code Fragment 9 shows the structure of a dataset in ShareGPT format.

```
[
  {"conversations": [
    {"role": "human", "content": "What is the capital of France?"},
    {"role": "gpt", "content": "The capital of France is Paris."}
  ]},
  ...
]
```

Code Fragment 9: The general Structure of a dataset in ShareGPT format

```
from unsloth import apply_chat_template

# Split the dataset
train_data_split = dataset.train_test_split(test_size=0.1, seed=seed)
train_data = train_data_split['train']
eval_data = train_data_split['test']

DEFAULT_SYSTEM_MESSAGE = "Du bist Stefan Zweig, ein bekannter deutscher
    Philosoph und Schriftsteller."

# Apply the default DeepSeek-R1 chat template to datasets
train_data = apply_chat_template(
    train_data,
    tokenizer = tokenizer,
    default_system_message = DEFAULT_SYSTEM_MESSAGE,
)

eval_data = apply_chat_template(
    eval_data,
    tokenizer = tokenizer,
    default_system_message = DEFAULT_SYSTEM_MESSAGE,
)
```

Code Fragment 10: Splitting the dataset into training and evaluation sets and tokenizing them

Once the dataset has the correct format, it is split into training and evaluation sets and tokenized (Code Fragment 10). Additionally, a default system message is added to provide the LLM with contextual information. This system message is also used during inference.

```

from trl import SFTTrainer
from transformers import TrainingArguments

trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = train_data,
    eval_dataset = eval_data,
    dataset_text_field = "text",
    max_seq_length = param_max_seq_length,
    dataset_num_proc = 2,
    packing = False,
    args = TrainingArguments(
        per_device_train_batch_size = param_batch_size,
        gradient_accumulation_steps = param_gradient_accumulation_steps,
        warmup_steps = 100,
        num_train_epochs = param_epochs,
        learning_rate = 2e-4,
        [...]
        metric_for_best_model = "eval_loss",
        save_strategy = "epoch",
        evaluation_strategy = "epoch",
    ),
)
trainer_stats = trainer.train()
model.save_pretrained_gguf("model", tokenizer)

```

Code Fragment 11: The training process

After model, tokenizer and datasets are prepared, the training begins. Code Fragment 11 shows an excerpt of the training code. The "Transformer Reinforcement Learning" library features the "Supervised Fine-Tuning Trainer" class, which can be used directly with its *trainer.train()* function to fine-tune an LLM.

The trainer's parameters are adjusted and, additionally, the *transformers.TrainingArguments* class is used to further specify training hyperparameters like learning rate, batch size and evaluation strategy.

After the training has finished, the best checkpoint according to the evaluation dataset is saved to disk.

5.3 Benchmarks & Metrics

This final package contains code for extensive benchmarks comparing all different models as well as code for the three metrics used for evaluating model performance:

1. **SemScore**: This metric measures the semantic similarity between two input strings by embedding them and calculating their dot product. This score has proven to be a good metric of response similarity in other papers [6].
2. **LangScore**: This metric checks if the response is in the correct language, in this case, German.
3. **Word Overlap**: This metric simply calculates the overlap of two input strings.

Code Fragment 12 shows how the three metrics are implemented in Python.

```
import langdetect
import difflib

def get_sem_score(input1, input2):
    embedding1 = embed(input1)
    embedding2 = embed(input2)
    return cosine_similarity(embedding1, embedding2)

def get_lang_score(input):
    languages = langdetect.detect_langs(input)

    de_probability = 0.0
    # Exit loop once probability for 'de' is found
    for lang_obj in languages:
        if lang_obj.lang == "de":
            de_probability = lang_obj.prob
            break
    return de_probability

def get_overlap_score(input1, input2):
    sm = difflib.SequenceMatcher(None, input1, input2)
    return sm.ratio()
```

Code Fragment 12: The three metrics used for qualitative evaluation

6 Experimental setup

In this section, the experimental setup will be detailed and results will be evaluated.

6.1 Examined DeepSeek variants

In total, 8 different variants based on DeepSeek have been examined:

- **DeepSeek-R1:8b and DeepSeek-R1:14b** - These models are the base models as provided by the DeepSeek-AI research team. These two models serve as a control to determine the performance of smaller base models only augmented with a small system prompt ("You are Brother David Steindl-Rast. [...]")
- **DeepSeek-R1:8b+RAG and DeepSeek-R1:14b+RAG** - These two models work by applying RAG using the Retrieval System of the original chatbot to the base models.
- **David-8b and David-14b** - These models are fine-tuned using 3600 input-output pairs generated by DeepSeek-R1:14b+RAG. The models have been fine-tuned for 5 epochs, with the best checkpoint being chosen.
- **David-OpenAI and David-OpenAI+RAG** - David-OpenAI is an instance of DeepSeek-R1:14b trained on 3600 input-output pairs generated by the original OpenAI chatbot, again fine-tuned for 5 epochs, with the best checkpoint being chosen. David-OpenAI+RAG aims to then further apply RAG to this LLM.

Additionally, the original, OpenAI and RAG based chatbot was used as a benchmark comparison. In the following chapters, this original OpenAI chatbot's response will be referred to as the "control response"

6.2 Benchmark Tests

For the benchmark tests, the original OpenAI model is presented with a list of 20 questions to each of which a response is generated. These question-answer pairs are then stored as a list of tuples. Code Fragment 13 shows the structure of the benchmark list.

Each of the DeepSeek variants is then also presented the same list of questions and is prompted to generate five responses per question, saving the average of each metric per question to a CSV file. Table 2 illustrates the structure and some selected information of the CSV file.

```
prompt_list = [
    ("Hallo, wer bist du?", "Ich bin eine Stimme, die [...]"),
    (...)
]
```

Code Fragment 13: The structure of the benchmark list

model	prompt	...	total_time	...	sem_score
deepseek-r1:8b	Hallo [...]	...	3.304	...	0.637
...	...	...	...	...	...
david-8b	Was ist [...]	...	2.377	...	0.910
...	...	...	...	...	...
david-openai+rag	Hallo [...]	...	3.464	...	0.825

Table 2: Example section taken from the generated CSV file

Additionally, the best and worst responses for each question, according to their semScore, are saved to aid in manually reviewing model performance.

6.3 Metrics for performance evaluation

The test results are being interpreted based on these metrics:

Time-based metrics:

- *RAG time*: The time it takes to receive an answer from the RAG service.
- *Prompt evaluation time*: The time it takes to process the input prompt, taken directly from the inbuilt Ollama metrics.
- *Response time*: The time it takes to generate the output tokens, taken directly from the inbuilt Ollama metrics.
- *Total time*: The total time it takes to receive a response from the LLM.

Token-based metrics:

- *Prompt token count*: The amount of tokens of the input, mainly used to monitor RAG overhead.
- *Response token count*: The amount of tokens of the LLM response.

Quality-based metrics:

- *SemScore*: The cosine similarity between LLM response and control response.
- *Word Overlap*: The amount of overlap between the LLM response and the control response, calculated using the *difflib* library.
- *Language Score*: The probability of a model response being German, as classified by the *langdetect* library.
- *Manual comparison*: Additionally, the LLM and control responses are analyzed to identify more nuanced and stylistic similarities.

7 Results

In this chapter, research results will be presented and analyzed.

7.1 Time and Token analysis

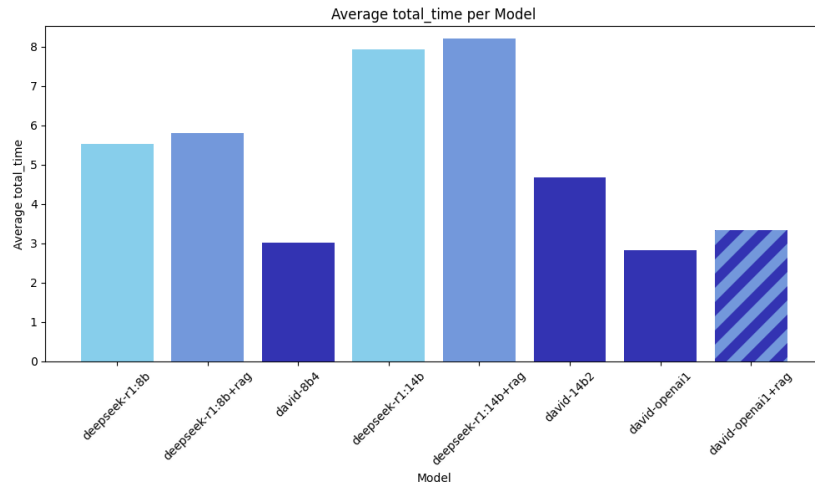


Figure 2: The average total response time per model

Figure 2 shows the average total time, meaning RAG, prompt evaluation and response generation time per model. A few trends can be seen in the graph:

1. 8b models are generally faster than their 14b counterparts
2. RAG only adds a small amount of overhead to the total time
3. The fine-tuned models are faster than the base or RAG models.
4. David-OpenAI, the 14b DeepSeek variant trained on OpenAI responses is even as fast as the fine-tuned 8b variant.

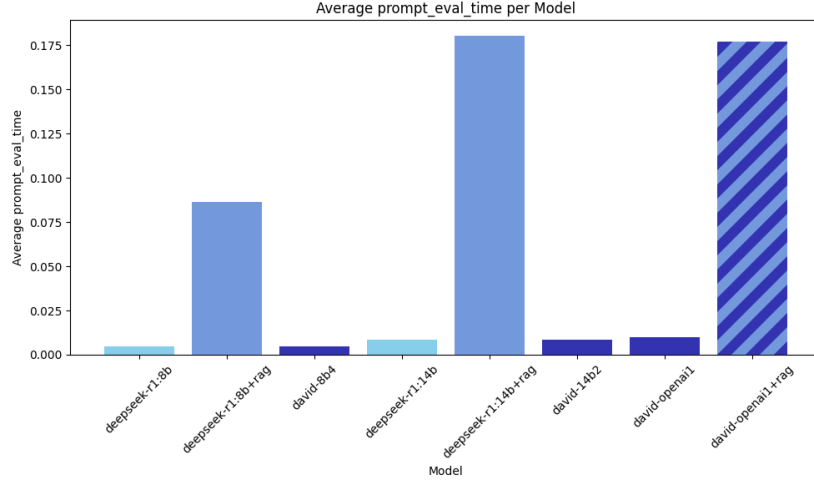


Figure 3: The average prompt evaluation time per model

Average Time Distribution (RAG vs Prompt Eval vs Response Generation)

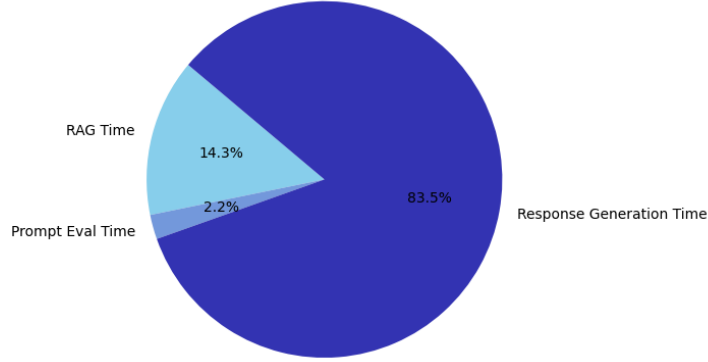


Figure 4: Distribution of RAG, prompt evaluation and response generation time in RAG-augmented models

The faster inference time observed with DeepSeek-8b-derived models is straightforward to explain: fewer weights and neurons directly translate to a reduced computational load, enabling quicker processing and faster responses.

Surprisingly, Retrieval Augmented Generation appears to have only a minor impact on total processing time, despite RAG-augmented prompts being significantly longer (typically over 2000 tokens) compared to non-RAG prompts (usually 30 to 70 tokens). Figure 3 shows that the longer prompt length does indeed have a

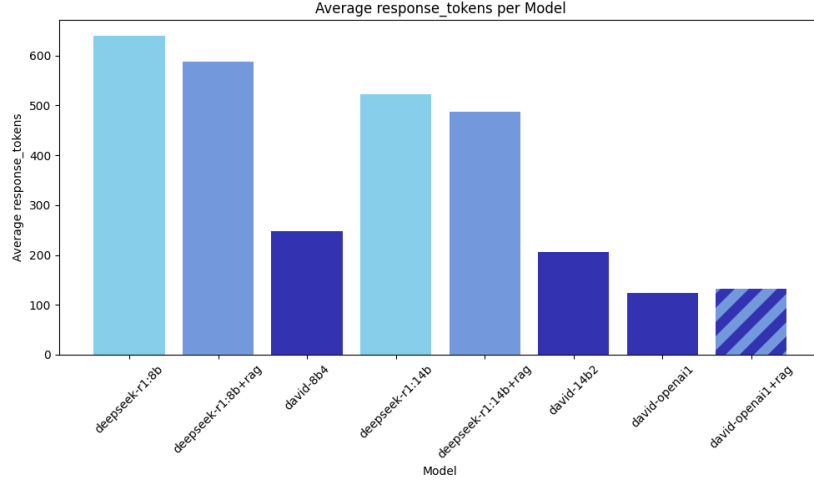


Figure 5: Average response tokens per model

noticeable impact on prompt evaluation time. However, this impact is negligible on the overall process, as prompt evaluation time is significantly shorter than response generation time (Figure 4), rendering RAG’s influence, beyond network latency, minimal.

Another interesting finding is visible in Figure 5, which shows the average response tokens per model. While the untrained DeepSeek-R1:8b and its RAG variant (along with their 14b counterparts) seem to generate a similar amount of response tokens, all trained models consistently produce significantly less output.

This difference can be attributed to the training data: The `<think>...</think>` block, which is characteristic of DeepSeek’s typical output, was excluded during the fine-tuning of the LLM. Consequently, over the course of training, the LLM learned to completely omit the generation of this ”think” block. The same is true for the 14b variant trained on OpenAI output, which does not feature a ”think” block altogether.

Figure 6 illustrates the time needed for response generation per model. It can again be noticed that smaller models, as well as models generating fewer output tokens, are faster.

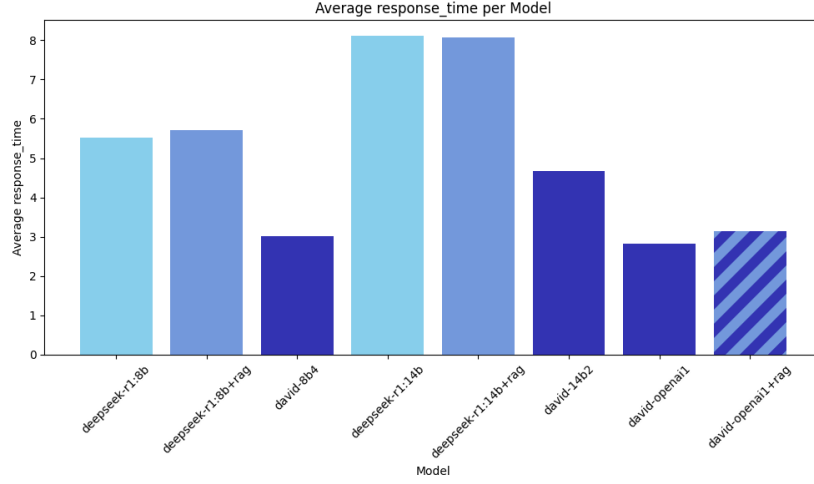


Figure 6: Average response time per model

7.2 Quality-based analysis

Since an important part of this work is not only model performance, but also the quality of generated output, all generated responses were also evaluated with the quality-based metrics featured in chapter 6.3.

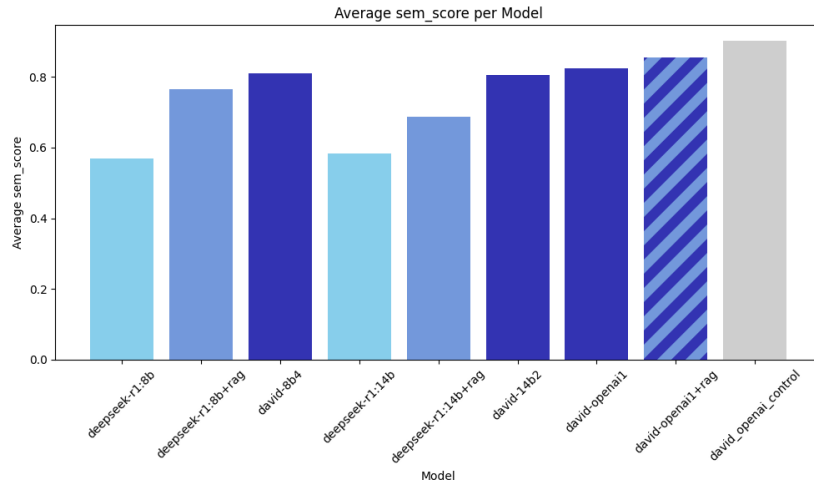


Figure 7: Average semScore across all prompts per model

Figure 7 shows the average semScore per model. It is clearly noticeable that the RAG chatbots perform slightly better than their respective base variants, with the fine-tuned models being even better. The gray bar on the right is the control,

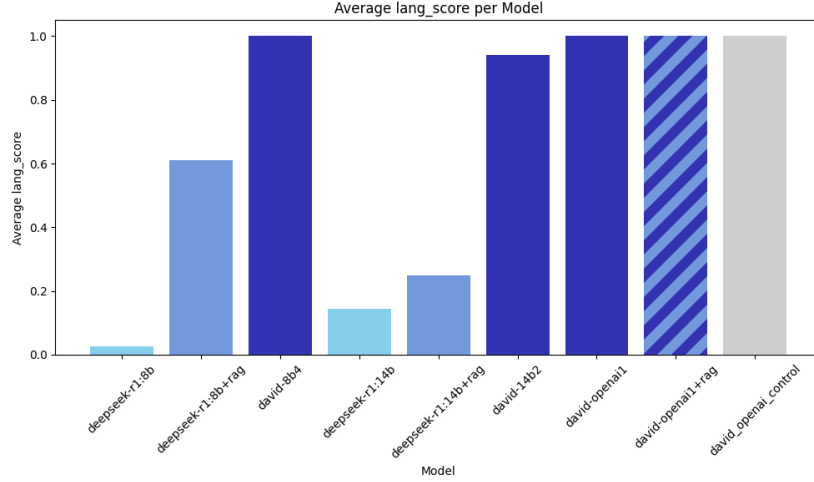


Figure 8: Average language score across all prompts per model

meaning that two OpenAI responses to the same question on average have a semantic similarity of about 0.8, a bar which all fine-tuned models nearly reach.

Figure 8 shows the average language score (langScore) per model across all prompts. An average langScore of 1.0 means that all responses were in German, while a langScore of 0.0 means that no response was in German.

It is immediately clear that the four untrained DeepSeek variants struggle with language mixing, with scores ranging from nearly 0.0 to about 0.6. The fine-tuned models on the other hand perform exceptionally well, with almost all achieving a langScore very close to 1.0.

The final measurable quality metric is the word overlap. Figure 9 again shows a clear correlation between training and high word overlap, with the two models fine-tuned with OpenAI data again outperforming the other models.

In addition to these measurable metrics, a manual analysis of all models was performed, which yielded the following additional findings:

- All untrained models suffer from language mixing, often answering in English or switching to Chinese mid-sentence.
- RAG-augmented base models work well, but responses do have a "style" not similar to the control OpenAI chatbot.

- Without RAG, safeguards are not properly in place and the models discuss topics they should normally not talk about.
- Fine-tuning leads to very convincing results, but quotes or other knowledge is often hallucinated by the LLM.
- The DeepSeek-R1:14b variant fine-tuned using OpenAI data, while still making up facts and quotes, copies the OpenAI style of writing convincingly.

The following responses show how convincingly the fine-tuned David-OpenAI learned to mimic the original chatbot:

Prompt: Was bedeutet Dankbarkeit für dich?

Original Chatbot:

Dankbarkeit ist mehr als nur Höflichkeit; sie ist eine Lebenshaltung. Sie lässt uns das Leben als Gabe erkennen und schafft Freude und Sinn in jedem Augenblick. Auch im Unglück schenkt die Dankbarkeit uns Wachstum und Vertrauen ins Leben. "Es geht gar nicht um das Danke sagen. Es geht um eine Lebenshaltung der Dankbarkeit." Wie kannst du heute einen Moment finden, um das Geschenk des Lebens bewusst als solches zu empfangen?

David-OpenAI:

Dankbarkeit ist mehr als höflich zu sein; sie ist eine Haltung des Herzens, ein bewusster Lebensvertrauen, das alles durchdringt. Sie führt uns ins Jetzt und lässt den Sinn von allem klar werden. Dankbarkeit erinnert uns daran: Es gibt Geschenke, die wir nie verdient haben und doch erhalten - ein Grund zur Freude in jedem Augenblick. Wie kannst du heute im Kleinen eine kleine Handlung der Dankbarkeit üben?

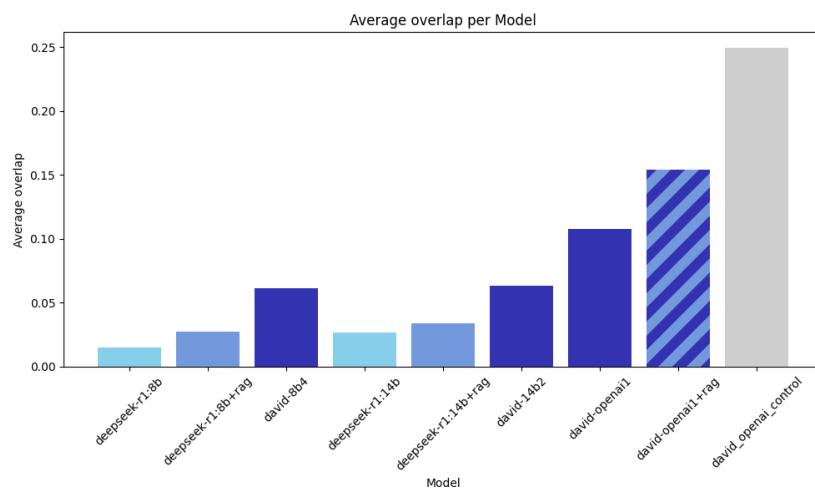


Figure 9: Average word overlap across all prompts per model

7.3 Analysis of the results

This chapter accumulates the findings presented in the preceding sections, drawing conclusions regarding quality and performance of the evaluated models.

Performance Insights

The time and token analysis reveals clear patterns regarding model efficiency. As expected, model size is a primary determinant of inference speed, with untrained 8b models consistently outperforming their 14b counterparts.

A particularly noteworthy finding is the minimal overhead introduced by Retrieval Augmented Generation on total processing time. Despite RAG-augmented prompts being significantly longer, token generation dominates the processing time.

Lastly, fine-tuning as done in this work improves model speed, albeit by reducing output length by omitting the DeepSeek specific "think" block.

Quality Evaluation

The quality-based analysis highlights the impact of fine-tuning on response quality, particularly concerning language consistency and semantic similarity. The results clearly show that untrained DeepSeek variants suffer from significant language mixing, mostly answering in English and sometimes even Chinese, even when directly prompted to answer in German. In stark contrast, the fine-tuned models perform exceptionally well, almost universally achieving langScores close to 1.0.

Regarding semantic similarity (semScore), the results show that RAG chatbots perform slightly better than their base variants, and fine-tuned models further improve upon this. Notably, the fine-tuned models nearly reach the control semScore of approximately 0.8, which represents the semantic similarity between two OpenAI responses to the same question. This suggests that the fine-tuning on RAG data significantly enhances the model's ability to generate semantically relevant and similar outputs. The word overlap analysis further supports this, showing a clear correlation between training and higher word overlap, with OpenAI-trained models again outperforming others.

The manual analysis provides further qualitative insights:

- Untrained models not only exhibit language mixing but also lack proper safeguards, occasionally discussing topics they should typically avoid.

- RAG-augmented base models, while improving semantic relevance, still produce responses with a distinct style that differs from the control OpenAI chatbot.
- Fine-tuning leads to very convincing results in terms of style and language, as exemplified by the DeepSeek-R1:14b variant fine-tuned with OpenAI data. This model convincingly mimics the OpenAI style of writing. However, a critical drawback of fine-tuning is the tendency for models to hallucinate facts and fabricate quotes. This trade-off between stylistic mimicry and factual accuracy is a significant consideration.

8 Conclusion

This thesis has investigated and compared the two major strategies for adapting the output of large language models, by applying RAG and fine-tuning using distilled DeepSeek-R1 models as a practical foundation. By attempting to replicate the behavior of the existing OpenAI-powered Brother David Bot on consumer hardware, the project evaluated not only the feasibility of such a task, but also the relative performance and trade-offs of both approaches.

The results of extensive benchmarking demonstrate that fine-tuned DeepSeek models, particularly those trained on data generated by the original OpenAI chatbot, were able to closely mimic its writing style and language. They significantly outperformed base models in both semantic similarity and language consistency. Moreover, fine-tuning led to faster inference times, largely due to shorter responses and the omission of the `<think>...</think>` block.

RAG, on the other hand, provided a flexible and non-invasive method for updating and enriching chatbot responses without modifying the model weights. Although the added retrieval step introduces slight latency and prompt inflation, its impact on total processing time was minimal. RAG’s ability to integrate external knowledge dynamically remains a key advantage, especially in contexts like chatbots where domain-specific information is required.

However, neither method is without its drawbacks: Fine-tuned models may hallucinate information and lack mechanisms for implementing guardrails for unwanted questions. RAG models, while accurate, may lack stylistic consistency, failing to accurately “mimic” the original chatbots style of writing.

In summary, this thesis shows that both approaches can be valuable tools in chatbot development, each excelling in different domains. For applications requiring high stylistic similarity and local deployment, fine-tuning proves advantageous. For dynamic, knowledge-rich scenarios, RAG offers more flexibility. The combined use of both methods may, in many cases, present the most robust and scalable solution.

References

- [1] Bibliothek David Steindl-Rast. <https://www.bibliothek-david-steindl-rast.ch/>. Accessed: 2025-06-30.
- [2] Brother David Bot. <https://bruderdavid.bot/de/>. Accessed: 2025-06-30.
- [3] Ollama Homepage. <https://ollama.com/>. Accessed: 2025-06-30.
- [4] Transformer Explainer. <https://poloclub.github.io/transformer-explainer/>. Accessed: 2025-06-14.
- [5] Unsloth Homepage. <https://unsloth.ai/>. Accessed: 2025-06-30.
- [6] Ansar Aynetdinov and Alan Akbik. Semscore: Automated evaluation of instruction-tuned llms based on semantic textual similarity. *arXiv preprint arXiv:2401.17072*, 2024.
- [7] Damai Dai, Chengqi Deng, Chenggang Zhao, RX Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Yu Wu, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.
- [8] Nan Du, Yanping Huang, Andrew M Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, et al. Glam: Efficient scaling of language models with mixture-of-experts. In *International conference on machine learning*, pages 5547–5569. PMLR, 2022.
- [9] Jindong Gu, Zhen Han, Shuo Chen, Ahmad Beirami, Bailan He, Gengyuan Zhang, Ruotong Liao, Yao Qin, Volker Tresp, and Philip Torr. A systematic survey of prompt engineering on vision-language foundation models. *arXiv preprint arXiv:2307.12980*, 2023.
- [10] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [11] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

- [12] Zhuang Liu, Wayne Lin, Ya Shi, and Jun Zhao. A robustly optimized bert pre-training approach with post-training. In *China national conference on Chinese computational linguistics*, pages 471–484. Springer, 2021.
- [13] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.
- [14] Guiyao Tie, Zeli Zhao, Dingjie Song, Fuyang Wei, Rong Zhou, Yurou Dai, Wen Yin, Zhejian Yang, Jiangyue Yan, Yao Su, et al. A survey on post-training of large language models. *arXiv preprint arXiv:2503.06072*, 2025.
- [15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [16] Xiao-Kun Wu, Min Chen, Wanyi Li, Rui Wang, Limeng Lu, Jia Liu, Kai Hwang, Yixue Hao, Yanru Pan, Qingguo Meng, et al. Llm fine-tuning: Concepts, opportunities, and challenges. *Big Data and Cognitive Computing*, 9(4):87, 2025.